Interactive API endpoint directory: test authentication flows, inspect payload schemas, and simulate error responses live

Computer
System migration runbook: sort legacy monolith tables to microservices, map foreign-key dependencies, and track cutover steps
Computer
Walk me through the assignments adapter code
Test the adapter with a mock database run
Generate the zero-downtime migration steps
provide everything nessarcy and required to turn everything here green

To turn every row in your screenshot green, Semester needs completed workflows, verified releases, explicit authority, and operational evidence—not just deployed screens or updated status badges.

I built a concrete completion kit and ran an assignments workflow against a disposable SQLite mock database. All 15 mock tests passed. I also created an interactive API/error simulator, a sortable migration tracker, and a workbook covering all 16 domains in your screenshot.

No production database, repository, deployment, or activation setting was changed.

1. What is ready now

Three deliverables accompany this response:

Deliverable	What it does
API and cutover lab	Simulates authentication outcomes and error responses, displays selected payload schemas, and sorts imported table/cutover records
All-domains green register	Tracks each screenshot domain's owner, implementation, tests, deployment, authority decision, and activation evidence
Completion and mock-test kit	Includes runnable JavaScript reference, SQLite backend, tests/results, PostgreSQL metadata queries, and migration runbook

Important boundaries

- The API lab is an offline interactive simulation. It does not authenticate against Semester or send live requests.
- The SQLite run tests a separate JavaScript behavioral reference, not the previously generated TypeScript adapter.
- The migration inventory query is supplied but was not executed against a live database.
- The workbook does not automatically approve a domain when evidence fields are filled.

2. Define green precisely

Your screenshot shows six distinct maturity columns. They should remain independent.

Column	Required green condition
Designed	Reviewed workflow, role, data, authority, interface, accessibility, and failure-state specifications
Implemented	Real frontend and backend complete the workflow without mock success or missing persistence
Tested	Positive, negative, authorization, concurrency, recovery, and accessibility results tied to the release
Deployed	Matching application, service, schema, and configuration versions are running and observed
Institution-authorized	Explicit approval for the institution, domain, data, roles, and intended operations
Production-enabled	Server-owned activation is enabled and the authorized end-to-end journey has been verified

For company-only or personal-only capabilities, institution authorization may be legitimately "not applicable"—but that requires a documented scope decision, not a fabricated "Yes."

The project release-gate catalog requires 22 completion checks, including tenant scope, permissions, source authority, accessibility, security, monitoring, tests, rollback, release evidence, and ownership. It explicitly identifies its own catalog as design-level until implementation evidence exists. [1]

The central rule is **evidence**: every green decision must point to a release-bound artifact, accountable reviewer, and revalidation condition.

3. Requirements for all 16 domains

The workbook contains these work packages and evidence fields.

Domain	Necessary completion work	Required acceptance journey
Identity and onboarding	Memberships, account lifecycle, recovery, delegation, sessions, offboarding	Sign in → correct workspace → two-device sync → revoke/offboard → access denied
Planning and degree	Versioned requirements, course relationships, approved planning rules	Create plan → check requirements → explain exceptions → preserve official/estimated distinction
Registration	Approved institutional connection, current eligibility, guarded commands	Prepare → confirm → execute → receipt; reconcile uncertain outcomes

Domain	Necessary completion work	Required acceptance journey
Courses and teaching	Faculty/TA section scopes, authoring, accessible publication	Author → preview → publish → student access → archive
Assignments and work	Versioned assignments, durable submissions, private-work separation	Submit → receipt → review; retry without duplicate effects
Study and tutoring	Trusted materials, course AI rules, tutor access, integrity boundaries	Study from approved sources → practice → seek help → respect restrictions
Grades and progress	Working/official distinction, review, release, corrections, passback	Propose → approve → release → reconcile → correct with history
Advising and support	Assigned caseloads, staff routes, cases, referrals, escalation	Request help → scoped staff review → referral → resolution
Career and portfolio	Student-controlled evidence, verification, employer sharing	Create evidence → publish selected material → verify → revoke/export
Campus and community	Membership, events, moderation, reporting, appeals	Join → participate → block/report → review → appeal
Housing, dining, services	Office-owned requests, eligibility, service status	Request → office action → status → resolution
Money and finance	Approved ledger/provider boundaries, approvals, settlement	Authorized transaction → receipt → reconciliation → refund/dispute
Institution OS	Tenant configuration, SSO/SCIM, grants, sources, control plane	Provision → configure → observe → review access → offboard
Company OS	Operational workflows and reconciled reporting	Close period → approve metrics → publish scoped board package
Developer and partner	Scoped APIs, contracts, sandbox, app review, revocation	Authorize integration → test scope → operate → revoke
Governed AI	Provider approval, gateway, policy, sources, budgets, bounded tools	Retrieve permitted evidence → respond → confirm allowed action → audit

The capability matrix also distinguishes ordinary read/write capabilities from high-impact commands, including grades and registration. Those operations require stronger execution and human-authority controls.[2]

4. Assignments adapter walkthrough

The intended adapter sequence is:

```
Trusted identity and tenant
    → current institutional approval
    → upstream section/object authorization
    → connection read/write permissions
    → record, action, fields, and version validation
    → consequence review
    → explicit confirmation
    → atomic authorized write
```

```
    → bound receipt or uncertain outcome
    → authoritative reconciliation
```

Why each step matters

1. Tenant check
   Prevents a valid identity in one workspace from selecting another institution's records.
2. Current approval
   Prevents an expired or revoked connection from continuing to operate.
3. Object authorization
   A recognized role does not establish enrollment, ownership, or delegated access to a particular assignment.
4. Version validation
   Prevents execution against a record that changed after review.
5. Side-effect-free review
   Reading and abandoning a preview must not create a submission or reserve anything.
6. Atomic write boundary
   Authorization, version checks, and durable idempotency must hold where the backend actually writes.
7. Uncertain-outcome handling
   A connection failure after submission may mean the write succeeded. Do not automatically submit again.
8. Reconciliation
   Look up the operation and return its authoritative result without repeating its effects.

5. Mock database test results

I executed 15 tests using Node's built-in test runner and a Python SQLite worker.

Test group	Observed result
Review without mutation	No submission created
Successful commit	One submission created
Same key and payload replay	Same receipt; no duplicate
Same key with changed payload	Rejected; no additional write
Cross-tenant and wrong-actor access	Denied
Grant revoked after review	Commit denied
Stale version	Rejected before write
Invalid fields or action	Rejected
Approval revoked	Operation blocked
Read-only	Writes blocked; permitted lookup retained

Test group	Observed result
Connection loss after durable commit	Reconciliation found the completed submission
Unknown operation	Remained unresolved
Concurrent duplicate requests	One submission created in this mock

All 15 passed; none were skipped.

What this does not prove

The worker serializes database calls. Concurrent Node requests therefore test duplicate handling in this mock, not distributed multi-worker concurrency.

This also does not verify the prior TypeScript patch, production PostgreSQL policies, real institutional authorization, upstream vendor behavior, or full-platform readiness.

6. Interactive API directory

The lab lets you:

- Select one of the fourteen previously inspected gateway routes.
- Choose a synthetic missing, rejected, or authorized identity.
- Inspect selected action payload schemas.
- Simulate 400, 401, 403, 409, 410, 429, 502, and 503 responses.
- Observe why an uncertain outcome requires reconciliation.

It accepts no real credentials and makes no network requests.

Schemas not verified in the inspected contracts are explicitly marked unverified. It does not invent complete intelligence or domain payload schemas.

7. Table dependencies and service boundaries

You do not need to split every table into a microservice to achieve green. First establish clear domain ownership and reliable shared boundaries. Extract services where there is a justified isolation, scaling, or operational need.

For migration, incrementally transform a domain while retaining the legacy path for coexistence and rollback, rather than attempting a simultaneous ecosystem rewrite. That is the structure described by the strangler-fig approach. [3]

The kit includes PostgreSQL metadata queries for:

- Tables and database owners.
- Row-level-security flags.
- Foreign-key relationships.
- Constraint definitions and validation status.

Those queries do not capture application references, views, functions, triggers, jobs, storage, or event dependencies. Those need separate inspection.

Dependency decisions

For each actual table or transactional cluster, record:

```
Source schema/table
Domain owner
Incoming and outgoing dependencies
Data classification and authority
Current writer
Proposed target
Backfill and change-capture strategy
Cutover gate
Rollback mechanism
Owner and evidence
```

Do not drop cross-service foreign keys merely to make a diagram look decomposed. Preserve transactional clusters or introduce explicitly tested reference-validation and reconciliation mechanisms.

8. Availability-preserving migration steps

“Zero downtime” should be a tested target, not a guarantee.

1. Confirm target, executor, baseline, recovery, and approved thresholds.
2. Inventory schema and hidden dependencies.
3. Introduce a compatible routing/domain boundary.
4. Expand schema additively with a rehearsed lock plan.
5. Add suitable indexes concurrently where supported.
6. Backfill in checkpointed, throttled batches.
7. Keep one authoritative writer; project changes through a durable mechanism.
8. Shadow reads and reconcile data, permissions, deletions, and lag.
9. Validate constraints and canary reads.
10. Fence old writes and drain or reconcile outstanding operations.
11. Switch write authority for the approved domain.
12. Verify new writes and rollback behavior.
13. Retire legacy structures only after the rollback window and retention requirements permit.

PostgreSQL schema changes can acquire strong locks. `NOT VALID` can defer the initial scan for supported constraints, with later validation, but it does not eliminate all locking.[4]

Concurrent index creation avoids the normal write-blocking index build, but has its own execution and readiness considerations.[5]

If continuity cannot be demonstrated during a write-authority transition, use a disclosed maintenance window rather than claiming zero downtime.

9. Execution order to reach green

Use shared foundations once, then complete connected domain journeys:

```
Identity and isolation
    + policy, provenance, evidence, durable execution
        → institution configuration and approved sources
        → courses, assignments, registration, records
        → support, campus, career, finance
        → company and partner workflows
        → per-domain production activation
```

AI policy and data restrictions belong in the foundation—not as cleanup after enabling unrestricted AI.

For every row, the remaining launch decision requires a named owner, real backend contract, approved environment, applicable authority, executed tests, support and monitoring, recovery evidence, and an activation receipt. The provided kit makes those requirements executable and trackable; it does not disguise missing work by changing the table's colors.

⁂

1. The Main Semester design system (2) copy/docs/master/SEMESTER_RELEASE_GATE_CATALOG.md
2. The Main Semester design system (2) copy/docs/native-platform/NATIVE_CAPABILITY_MATRIX.md
3. https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-decomposing-monoliths/strangler-fig.html
4. https://www.postgresql.org/docs/current/sql-altertable.html
5. https://www.postgresql.org/docs/current/sql-createindex.html
6. https://aws.amazon.com/video/watch/e34606c07e4/
7. https://www.runxbuild.com/blog/postgres-add-column/
8. https://www.catio.tech/blog/strangler-fig-pattern
9. https://www.cubic.dev/agents/postgres-add-constraints-without-locking
10. https://dev.to/axeldlv/strangler-fig-migration-strategy-on-aws-17l0
11. https://squawkhq.com/docs/constraint-missing-not-valid
12. https://dba.stackexchange.com/questions/343791/do-you-need-not-valid-when-adding-new-foreign-key-column
13. https://stackoverflow.com/questions/76156967/postgresql-add-unique-constraint-to-existing-table-without-downtime-locking
14. https://pglocks.org/?pgcommand=CREATE INDEX
15. Screenshot-2026-10-11-at-4.30.30-PM.jpg
16. https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/strangler-fig.html
17. https://aws.amazon.com/blogs/architecture/seamlessly-migrate-on-premises-legacy-workloads-using-a-strangler-pattern/

18. https://www.citusdata.com/blog/2018/02/22/seven-tips-for-dealing-with-postgres-locks/